

A Simple Hybrid Event-B Model of an Active Control System for Earthquake Protection



Richard Banach and John Baugh

Abstract In earthquake-prone zones of the world, severe damage to buildings and life endangering harm to people pose a major risk when severe earthquakes happen. In recent decades, active and passive measures to prevent building damage have been designed and deployed. A simple model of an active damage prevention system, founded on earlier work, is investigated from a model based formal development perspective, using Hybrid Event-B. The non-trivial physical behaviour in the model is readily captured within the formalism. However, when the usual approximation and discretization techniques from engineering and applied mathematics are used, the rather brittle refinement techniques used in model based formal development start to break down. Despite this, the model developed stands up well when compared via simulation with a standard approach. The requirements of a richer formal development framework, better able to cope with applications exhibiting non-trivial physical elements are discussed.

1 Introduction

In earthquake-prone zones of the world, damage to buildings during an earthquake is a serious problem, leading to major rebuilding costs if the damage is severe. This is to say nothing of the harm to people that ensues if they happen to be inside, or near to, a building that fails structurally. One approach to mitigating the problem is to make buildings so robust that they can withstand the severest earthquake that may befall them; but this not only greatly increases cost, but also places limits on the size

R. Banach (✉)

School of Computer Science, University of Manchester, Oxford Road,
Manchester M13 9PL, UK

e-mail: richard.banach@manchester.ac.uk

J. Baugh

Department of Civil, Construction, and Environmental Engineering,
North Carolina State University, Raleigh, NC 27695-7908, USA

e-mail: jwb@ncsu.edu

© Springer Nature Switzerland AG 2020

A. Adamatzky and V. Kendon (eds.), *From Astrophysics to Unconventional Computation*, Emergence, Complexity and Computation 35,
https://doi.org/10.1007/978-3-030-15792-0_7

and shape of buildings, so that the desired robustness remains feasible with available materials.

In recent decades, an alternative approach to earthquake protection has been to use control techniques to dissipate the forces that reach vulnerable elements of a building by using control strategies of one kind or another. In truth, the first proposal for intervening in a building's ability to withstand earthquake dates back to 1870 (a patent filed at the U.S. Patent Office by one Jules Touaillon), but such ideas were not taken seriously till a century or so later.

One approach is to use passive control. In this approach massive members and/or damping mechanisms are incorporated into the building in such a way that their parameters and the coupling between them and the rest of the building are chosen just so that the destructive forces are preferentially dissipated into these additional elements, leaving the building itself undamaged.

An alternative, more recent approach, is to use active control. Since it is the amplitude and frequency of the vibrations that a building is subject to during an earthquake that determine whether it will sustain damage or not, damping earthquake vibrations by applying suitably designed counter vibrations to the building reduces the net forces that the building must withstand, and thus the damage it will sustain under a given severity of earthquake and given a specific standard of construction.

In [16, 20, 21, 31] there is a study of such an active control system for earthquake resistance for buildings. Ultimately, it is targetted at an experimental tall building of six stories. These papers investigate various aspects of verification for a system of this kind, based largely on timing considerations, which inevitably generate uncertainties due to equipment latencies. The approach to the system is rather bottom up in [20, 21]. The design is presented at a low level of detail with separate elements for the start of an action, the end of the action, and a synchronisation point within the action (as needed), with timings attaches to each element. Even for a simple system, this results in a huge state space. The focus of [20, 21] then becomes reduction of the state space size, showing no loss of behaviour via bisimulation. Following this, useful properties of the system model may be demonstrated using the smaller state space version.

In the present paper, we take an alternative route, going top down instead of bottom up, and using Hybrid Event-B (henceforth HEB) [11, 12] as the vehicle for doing the development. We work top-down, and for simplicity and through lack of space, we do not get to the low level of detail present in [20, 21]. In particular, we omit replication of subsystems, timing and fault tolerance (though we comment on these aspects at the end). As well as providing the contrast to the previous treatment, the present case study offers some novelty regarding the interaction of physical and digital behaviours (in particular, regarding continuous vs. impulsive physics, as treated within the HEB formalism), compared with other case studies, e.g. [5–7, 9, 10, 15].

The rest of this paper is as follows. In Sect. 2 we briefly overview control strategies for seismic protection for buildings, and focus on the active control principles that underpin this paper's approach. Section 3 has an outline of single machine HEB, for purposes of orientation. In Sect. 4 we present the simple dynamical model we develop, and its most abstract expression in HEB, and Sect. 5 presents an ideal but

completely unrealistic solution to the problem posed in the model. The next few sections develop and refine the original model in a less idealised way, bringing in more of the detailed requirements of a practical solution. The more detail we bring in, the greater the challenge to the usual refinement technique found in formal development frameworks, including HEB.

Thus Sect. 6 presents a first refinement towards a practical solution, while Sect. 7 engages more seriously with an ‘ideal pulse’ strategy for the active control solution. Section 8 pauses to discuss the issues for refinement that this throws up. Section 9 incorporates the discretization typically seen in practical engineering solutions, and also treats decomposition into a family of machines that reflect a more convincing system architecture, one resembling the approach of [20, 21]. Section 10 continues the discussion of issues raised for refinement and retrenchment by these development steps. Section 11 presents numerical simulation work showing that the theoretically based earlier models give good agreement when compared with solutions derived using conventional engineering approaches. Section 12 recapitulates and concludes.

2 Control Strategies for Earthquake Damage Prevention

Since mechanical prevention of earthquake damage to structures began to be taken seriously, a number of engineering techniques have been brought to bear on the problem [40]. These days this is a highly active field and the literature is large, e.g. [1, 2, 19].

In passive control [17], decoupling of the building from the ground, and/or the incorporation of various additional members, are used to ensure that the forces of an earthquake do not impinge on the important building structure. Passive approaches are often used to protect historical buildings in which major re-engineering is impractical. One disadvantage of the passive approach is the potential transverse displacements relative to the ground that the protected building may undergo. If, with respect to an inertial frame, the building stays still, and the ground moves by 20 cm, then the relative movement of the building is 20 cm. This may not be practical.

In alternative approaches the engineering compensation is more active. Active approaches (such as the one we will pursue in more detail below) have the compensation mechanism trying to actively counter the forces imparted by the earthquake in order to limit the amplitude of vibrations at the building’s resonant frequencies [33, 34]. One problem experienced by active prevention systems is that they may consume a lot of energy, which is expensive and undesirable. Another is that if one is unlucky, and the parameters of the earthquake fall in the wrong region (due to imprecise design, or error), then because an active system is injecting energy into the overall structure, it may actually make things worse, driving the overall structure into instability, perhaps because the injected energy is being introduced in phase rather than in anti-phase with the earthquake itself. An increasingly popular approach these days is semi-active control [22], in which the main aim is to intervene actively only in

the dissipative part of the building's response, decreasing energy costs and avoiding potential instabilities, for only a small decrease in performance.

In all of these active strategies for prevention of earthquake damage to buildings, the building contains a set of sensors which constantly monitor vibrations impinging on the building from the ground. The signals coming from these are analysed to differentiate between earthquake originated forces, and normal day to day vibrations caused by everyday activities in the building's surroundings. The latter are, of course, ignored.

The building also contains active members which can impart forces to the building structure. The aim of the active control is to impart forces to the building that counter the damaging forces coming from the earthquake, so that the net force that the building must withstand remains within its safe design parameters.

These days, these design aims are achieved using a sophisticated control engineering approach. Many strategies have been tried, but among the most popular currently is to use a LQG (Linear Quadratic Gaussian) strategy for designing a nominal controller, which is then modulated by clipping extreme values. This approach is based on a sophisticated formulation of noisy dynamics and its control via a Bayesian estimation of current and future behaviour. See e.g. [22] (or [28] for a simpler example). One consequence of this approach is some loss of direct contact between the control algorithm design and real time values, due to the use of L^2 estimates in the derivation of the controller. This is a disadvantage regarding direct correspondence with typical formal methods approaches, which are wedded exclusively to variable values in real time.

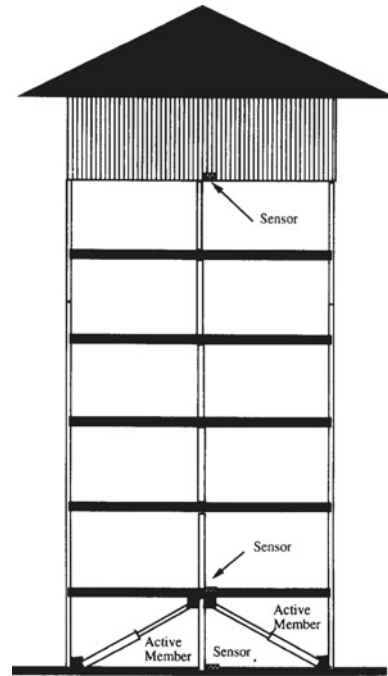
Our own study is based on the strategy used in [20, 21]. Figure 1, taken from [20, 21], gives a schematic outline of how active elements are disposed in an experimental building in Tokyo. There are sensors near the ground, and at the top of the building. The active members, which have to be capable of exerting significant force if they are to move significant aspects of the building, are found at the bottom of the building.¹

The technique by which the corrective forces are applied to the building is to have the active members impart a series of pulses to the core framework of the building. Of course, for this to be successful on the timescales of earthquake vibrations, there has to be accurate real time control, and an appropriate balance between the aggregated effect of the applied pulse series and the sensed vibrations coming from the earthquake. In contrast to the LQG approach, the technique used in [16, 20, 21, 29, 31] is based on real time monitoring of positions, velocities and accelerations in the building's structure, thus greatly facilitating a correspondence with conventional model based formal methods techniques (a point that emerges, though obviously quite indirectly, from remarks in [29]).

It is not our aim in this paper to get deeply embroiled in the detailed control engineering aspects of the problem. We leave that to other work. Instead, our aim

¹In sophisticated modern designs, active members are also found higher up the building, to counter vibration antinodes part way up a tall structure.

Fig. 1 A schematic of a building design, to be protected by an active earthquake damage prevention system.
From [20, 21]



is to take a top down approach to the implementation task, and to see how a HEB perspective can bring efficiencies and a degree of clarity to that. Accordingly, we next turn to HEB itself.

3 A Brief Outline of Hybrid Event-B

In this section we give an outline of Hybrid Event-B for single machines. In Fig. 2 we see a bare bones HEB machine, *HyEvBMch*. It starts with declarations of time and of a clock. In HEB, time is a first class citizen in that all variables are functions of time, whether explicitly or implicitly. However time is special, being read-only, never being assigned, since time cannot be controlled by any human-designed engineering process. Clocks allow a bit more flexibility, since they are assumed to increase their value at the same rate that time does, but may be set during mode events (see below). Variables are of two kinds. There are mode variables (like u , declared as usual) which take their values in discrete sets and change their values via discontinuous assignment in mode events. There are also pliant variables (such as x, y), declared in the PLIANT clause, which take their values in topologically dense sets (normally $\mathbb{R}$) and which are allowed to change continuously, such change being specified via pliant events (see below).

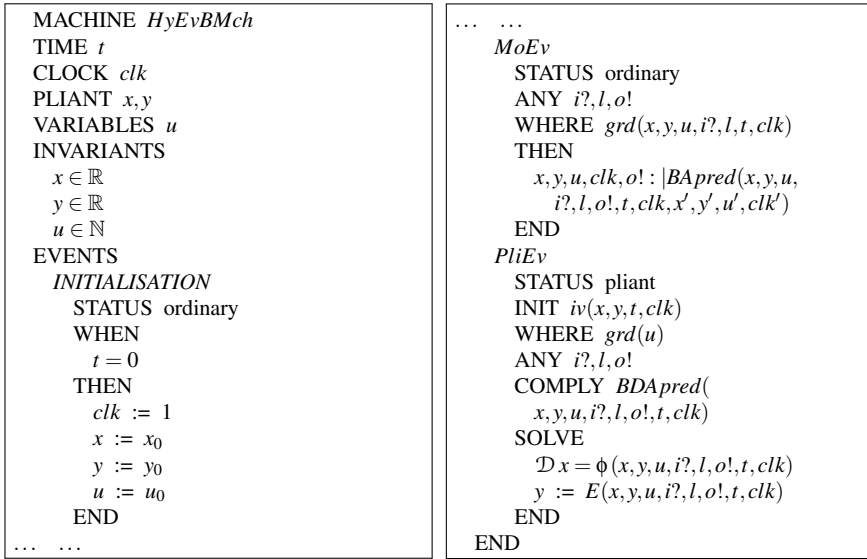


Fig. 2 A schematic Hybrid Event-B machine

Next are the invariants. These resemble invariants in discrete Event-B, in that the types of the variables are asserted to be the sets from which the variables' values *at any given moment of time* are drawn. More complex invariants are similarly predicates that are required to hold *at all moments of time* during a run.

Then we get to the events. The *INITIALISATION* has a guard that synchronises time with the start of any run, while all other variables are assigned their initial values in the usual way. As hinted above, in HEB, there are two kinds of event: mode events and pliant events.

Mode events are direct analogues of events in discrete Event-B. They can assign all machine variables (except time itself). In the schematic *MoEv* of Fig. 2, we see three parameters *i?, l, o!*, (an input, a local parameter, and an output respectively), and a guard *grd* which can depend on all the machine variables. We also see the generic after-value assignment specified by the before-after predicate *BApred*, which can specify how the after-values of all variables (except time, inputs and locals) are to be determined.

Pliant events are new. They specify the continuous evolution of the pliant variables over an interval of time. The schematic pliant event *PliEv* of Fig. 2 shows the structure. There are two guards: there is *iv*, for specifying enabling conditions on the pliant variables, clocks, and time; and there is *grd*, for specifying enabling conditions on the mode variables. The separation between the two is motivated by considerations connected with refinement.

The body of a pliant event contains three parameters *i?, l, o!*, (once more an input, a local parameter, and an output respectively) which are functions of time, defined over the duration of the pliant event. The behaviour of the event is defined

by the COMPLY and SOLVE clauses. The SOLVE clause specifies behaviour fairly directly. For example the behaviour of pliant variable y is given by a direct assignment to the (time dependent) value of the expression $E(\dots)$. Alternatively, the behaviour of pliant variable x is given by the solution of the first order ordinary differential equation (ODE) $\mathcal{D}x = \phi(\dots)$, where $\mathcal{D}$ indicates differentiation with respect to time. (In fact the semantics of the $y = E$ case is given in terms of the ODE $\mathcal{D}y = \mathcal{D}E$, so that both x and y satisfy the same regularity properties.) The COMPLY clause can be used to express any additional constraints that are required to hold during the pliant event via its before-during-and-after predicate $BDAPred$. Typically, constraints on the permitted range of values for the pliant variables, and similar restrictions, can be placed here.

The COMPLY clause has another purpose. When specifying at an abstract level, we do not necessarily want to be concerned with all the details of the dynamics—it is often sufficient to require some global constraints to hold which express the needed safety properties of the system. The COMPLY clauses of the machine's pliant events can house such constraints directly, leaving it to lower level refinements to add the necessary details of the dynamics.

Briefly, the semantics of a HEB machine is as follows. It consists of a set of *system traces*, each of which is a collection of functions of time, expressing the value of each machine variable over the duration of a system run. (In the case of *HyEvBMch*, in a given system trace, there would be functions for clk, x, y, u , each defined over the duration of the run.)

Time is modeled as an interval $\mathcal{T}$ of the reals. A run starts at some initial moment of time, t_0 say, and lasts either for a finite time, or indefinitely. The duration of the run $\mathcal{T}$, breaks up into a succession of left-closed right-open subintervals: $\mathcal{T} = [t_0 \dots t_1), [t_1 \dots t_2), [t_2 \dots t_3), \dots$. The idea is that mode events (with their discontinuous updates) take place at the isolated times corresponding to the common endpoints of these subintervals t_i , and in between, the mode variables are constant and the pliant events stipulate continuous change in the pliant variables.

Although pliant variables change continuously (except perhaps at the t_i), continuity alone still allows for a wide range of mathematically pathological behaviours. To eliminate these, we make the following restrictions which apply individually to every subinterval $[t_i \dots t_{i+1})$:

- I Zeno: there is a constant δ_{Zeno} , such that for all i needed, $t_{i+1} - t_i \geq \delta_{\text{Zeno}}$.
- II Limits: for every variable x , and for every time $t \in \mathcal{T}$, and for $\delta > 0$, the left limit $\lim_{\delta \rightarrow 0} x(t - \delta)$ written $\overrightarrow{x(t)}$ and right limit $\lim_{\delta \rightarrow 0} x(t + \delta)$, written $\overleftarrow{x(t)}$, exist, and for every t , $x(t) = \overleftarrow{x(t)}$. [N.B. At the endpoint(s) of $\mathcal{T}$, any missing limit is defined to equal its counterpart.]
- III Differentiability: The behaviour of every pliant variable x in the interval $[t_i \dots t_{i+1})$ is given by the solution of a well posed initial value problem $\mathcal{D}xs = \phi(xs \dots)$ (where xs is a relevant tuple of pliant variables and $\mathcal{D}$ is the time derivative). ‘Well posed’ means that $\phi(xs \dots)$ has Lipschitz constants which are uniformly bounded over $[t_i \dots t_{i+1})$ bounding its variation with respect to xs , and that $\phi(xs \dots)$ is measurable in t .

Regarding the above, the Zeno condition is certainly a sensible restriction to demand of any acceptable system, but in general, its truth or falsehood can depend on the system's full reachability relation, and is thus very frequently undecidable.

The stipulation on limits, with the left limit value at a time t_i being not necessarily the same as the right limit at t_i , makes for an easy interpretation of mode events that happen at t_i . For such mode events, the before-values are interpreted as the left limit values, and the after-values are interpreted as the right limit values.

The differentiability condition guarantees that from a specific starting point, t_i say, there is a maximal right open interval, specified by t_{MAX} say, such that a solution to the ODE system exists in $[t_i \dots t_{\text{MAX}})$. Within this interval, we seek the earliest time t_{i+1} at which a mode event becomes enabled, and this time becomes the preemption point beyond which the solution to the ODE system is abandoned, and the next solution is sought after the completion of the mode event.

In this manner, assuming that the *INITIALISATION* event has achieved a suitable initial assignment to variables, a system run is *well formed*, and thus belongs to the semantics of the machine, provided that at runtime:

- Every enabled mode event is feasible, i.e. has an after-state, and on its completion enables a pliant event (but does not enable any mode event).² (1)
- Every enabled pliant event is feasible, i.e. has a time-indexed family of after-states, and EITHER: (2)
 - (i) During the run of the pliant event a mode event becomes enabled. It preempts the pliant event, defining its end. ORELSE
 - (ii) During the run of the pliant event it becomes infeasible: finite termination. ORELSE
 - (iii) The pliant event continues indefinitely: nontermination.

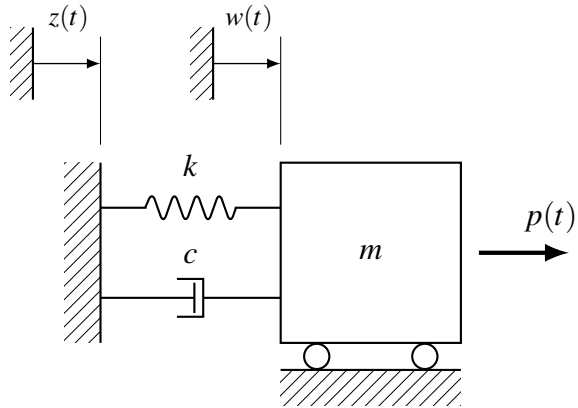
Thus in a well formed run mode events alternate with pliant events. The last event (if there is one) is a pliant event (whose duration may be finite or infinite).

We note that this framework is quite close to the modern formulation of hybrid systems. (See e.g. [27, 35] for representative formulations, or the large literature in the *Hybrid Systems: Computation and Control* series of international conferences, and the further literature cited therein.)

In reality, there are a number of semantic issues that we have glossed over in the framework just sketched. We refer to [11] for a more detailed presentation. Also, the development we undertake requires the multi-machine version of HEB [12]. Since the issues that arise there are largely syntactic, we explain what is needed for multi-machine HEB in situ, as we go along.

² If a mode event has an input, the semantics assumes that its value arrives at a time distinct from the previous mode event, ensuring part of (1) automatically.

Fig. 3 The simple mechanical model that the HEB development is based on, after [31]



4 Top Level Abstract Model of the Control System

As stated in Sect. 2, we do not get deeply embroiled in the detailed control engineering aspects of realistic active control in this paper. We base our treatment on the relatively simple strategy described in detail in [31].

Figure 3 shows the simple system investigated in [29, 31]. The model refers to the dynamics of a mechanical system with a single degree of freedom (SDOF). The building to be protected is modelled as a concentrated or lumped mass m and a structural system that resists lateral motion with a spring of stiffness k and a viscous damper with coefficient c .³ A force p is applied to the mass by the active control system. The effects of spring and damper depend on the relative position between a fiducial point in the building w , and another fiducial point in the earth z , i.e. on $x = w - z$. When $w = z = 0$, the spring is unstretched. Writing $\mathcal{D}$ for the time derivative, and defining $e \equiv \mathcal{D}^2 z$, the dynamics of w , expressed in terms of the relative displacement x , is thus controlled by:

$$m \mathcal{D}^2 x + c \mathcal{D} x + k x = p - m e \quad (3)$$

Since p is to be chosen by the system, e can be measured, and the other data are known, (3) yields a law that can be used to keep x within desired bounds.

The code for the top level model of the HEB development is in Fig. 4. At this level the system consists of a single machine, *ActConMch_0*. There are pliant variables x, p , which capture the model elements discussed above.

The *INVARIANTS* are rather basic at this stage. They declare the types of the variables, and one further non-trivial property. This property actually expresses the

³Idealizing a building by an equivalent SDOF system requires an assumption about its displaced shape and other details that are beyond the scope of the paper. The interested reader is directed to the methodology outlined by Kuramoto et al. [24], which is included in the current building design code used in Japan, as one example.

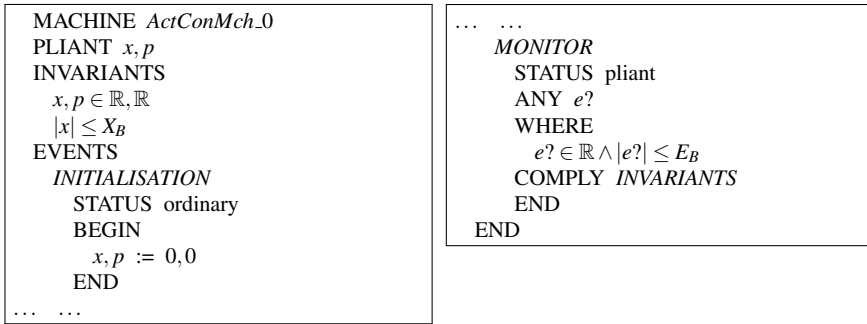


Fig. 4 A highly abstract model of the earthquake damage prevention active control system

key system requirement of the whole development, namely that the value of variable x stays within a range $-X_B \leq x \leq X_B$. Imposing it amounts to placing a limit on the lateral drift⁴ of a building structure, i.e., the horizontal displacement that upper stories undergo with respect to the base. This relative motion is resisted by the building's structural system, but under extreme events the internal deformations may be excessive, leading to structural damage and ultimately collapse of the building.

The *INITIALISATION* event sets all the variables to zero. Then there is the single actual event of the model: the *MONITOR* pliant event, which covers the continuous monitoring of the system when it is in the monitoring mode. Since this is the only mode in the model, it does not require a specific variable or value to name it.⁵

The definition of the *MONITOR* pliant event is trivial at this level of abstraction. It consumes its input $e?$ evidently corresponding to the relevant model element above. For future calculational tractability, $e?$ is assumed to be bounded by an explicit constant E_B . The event simply demands that the *INVARIANTS* are to be maintained. This is to be established in some as yet unspecified manner, since we postpone the more demanding details of the calculations involved in the control model we have introduced. Later on, its more precisely defined job will be to monitor the information coming from the sensors (i.e. to monitor w and its derivatives, and e), to calculate what response may be necessary, and to issue pulses to the actuators, as may be required, these being the embodiment of p . Of course, in reality, the monitoring will be done via a series of discrete events, resulting in a series of readings from the sensors, but for the next few models in the development, we will assume that this is all a continuous process.

⁴The International Building Code (IBC) requires that drift be limited in typical buildings to 1–2% of the building's height for reasons of both safety and functional performance.

⁵Of course, a more realistic model would contain modes for maintenance, and for other forms of partial running or of inactivity—to be used, presumably, only when the building is unoccupied.

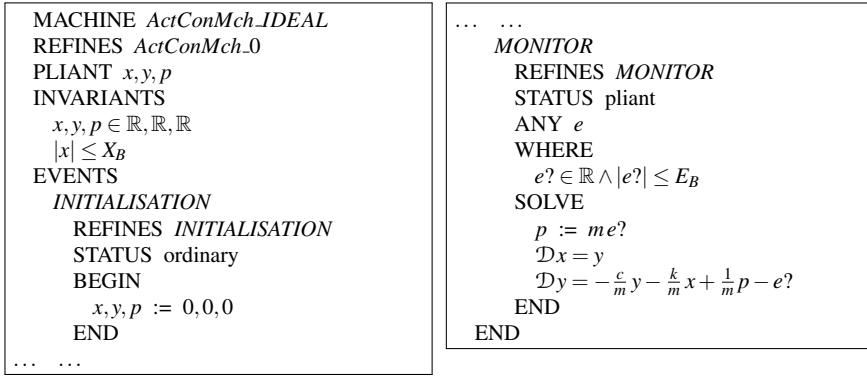


Fig. 5 Idealised refinement of the system

5 An Idealised Refinement: Miraculous ODE Behaviour

Figure 5 presents a somewhat idealised refinement of *ActConMch_0*. Rather than assume the desired effect is achieved nondeterministically, we introduce the control law (3) in the SOLVE clause of *MONITOR*. In order to do that we introduce a new variable y so that we can translate the second order (3) into the first order form stipulated by HEB. Having done that, we notice that we need merely to set p to $me?$ and the zero initialisation of x, y persists to a global solution satisfying the invariants. We are done! The building stands still under all admissible earthquake conditions!

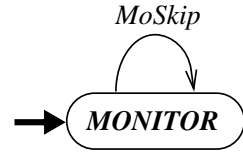
If only it were that simple. Unfortunately, it requires that p be chosen to mirror the instantaneous real time behaviour of $e?$ with complete precision, with no allowance for quantisation effects or for signal propagation delay in equipment. This is impractical in the real world. Accordingly, we abandon this route in favour of a more achievable development route.

6 A More Realistic Refinement: Achievable ODE Behaviour

The problem with the *MONITOR* of Fig. 5 is that it is already so precise that there is no way to backtrack to a more tolerant engineering model *while remaining within the restrictions of refinement theory*. In this section we have a different refinement of *ActConMch_0*, which allows some leeway for engineering imprecision.

In Fig. 6 we have a transition diagram representation of the system for this refinement. As before there is only one state, the one occupied by the *MONITOR* event, and there is now a mode event too, *MoSkip*, of which more later. The HEB code for the refinement is in Fig. 7.

Fig. 6 A transition diagram for the first refinement of the HEB model of the earthquake damage prevention system



The behaviour of the *MONITOR* event refines its previous incarnation by restricting the behaviour. As well as the input $e?$ there are now two locally chosen parameters, pp and e . The former allows values that match $e?$ imprecisely to be fed to the ODE system in the *SOLVE* clause (which is almost the same as in Fig. 5), while the latter permits the stipulation that $e?$ differs from a constant value (which may be chosen conveniently) by not too much during a *MONITOR* transition.

The *COMPLY* clause takes advantage of the fact that the solution to such linear constant coefficient inhomogeneous ODE systems is routine. See [4, 18, 32, 36] as well as a host of other sources. The first term is the homogeneous solution, primed by the initial values: $e^{\mathbf{A}(t-\mathfrak{t}_L)}[x(\mathfrak{t}_L), y(\mathfrak{t}_L)]^T$, where $\mathbf{A}$ is the companion matrix of the homogeneous part of the ODE system in the *SOLVE* clause, and $\mathfrak{t}_L$ refers, generically, to the start time of any runtime transition specified by the pliant event. The second term is the convolution $*$ over the interval $[\mathfrak{t}_L \dots t]$ between the homogeneous solution $e^{\mathbf{A}(s)}$ (with bound convolution variable renamed to s) and the inhomogeneous part $[0, \frac{1}{m}pp - e?]^T$. If the projection of all this to the x variable (written $.x$) achieves the desired bound, then the ODE system in the *SOLVE* clause establishes the desired

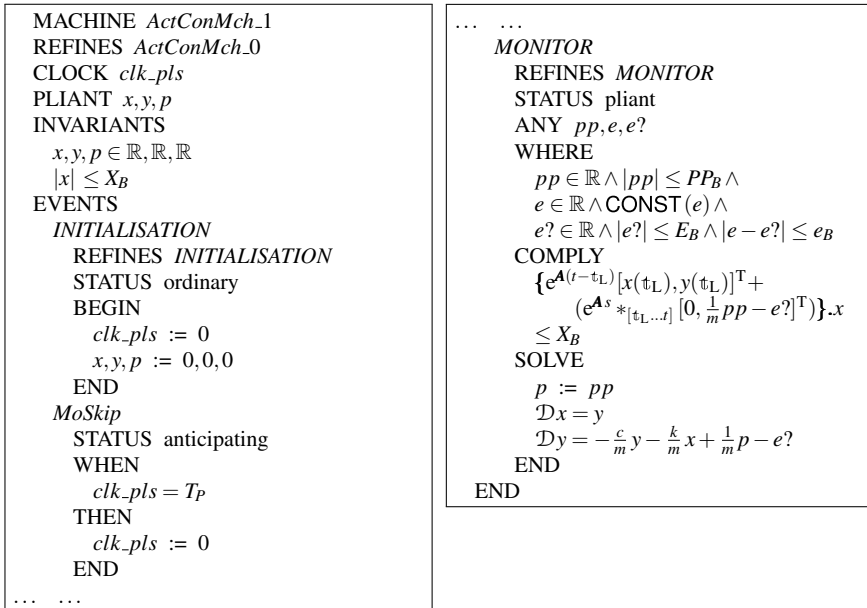


Fig. 7 First refinement of the system

invariant. The permitted imprecision between pp and $e?$ now makes this a practical proposition.

The mode event *MoSkip* interrupts the *MONITOR* event at intervals of T_P , after which *MONITOR* restarts. This permits the reassignment of the constant e in *MONITOR* at each restart. If the interval T_P is short enough it permits the choice of pp during each *MONITOR* transition to achieve the desired outcome.

The caption of Fig. 7 claims that *ActConMch_1* is a refinement of *ActConMch_0*. Indeed it is, although we do not describe the details of this here; see [11] for a more thorough account. We note though that: the invariants are not weakened; the new initialisation is evidently consistent with the old; the new behaviour of *MONITOR* evidently satisfies the previous definition; and the new mode event only updates a newly introduced (clock) variable. All of these are characteristics of HEB refinement.

7 Refining pp

The objective of the next refinement is to address the specific form of pp , bringing the design closer to engineering practice, and to [29, 31] in particular. For this we follow the detailed formulation in [19], which contains a wealth of detailed calculation. We make a conventional change of parameters: $\zeta = c/2\sqrt{km}$, $\omega_n = \sqrt{k/m}$, $\omega_D = \omega_n\sqrt{1 - \zeta^2}$. This change reduces the LHS of (3) to (m times):

$$\mathcal{D}^2x + 2\zeta\mathcal{D}x + \omega_n^2x \quad (4)$$

In terms of these quantities, the generic solution of the ODE system indicated above reduces to a Duhamel integral with specified initial values [19, 39]:

$$\begin{aligned} x(t - \mathfrak{t}_L) = & e^{-\zeta\omega_n(t-\mathfrak{t}_L)} \left[x(\mathfrak{t}_L) \cos(\omega_D(t - \mathfrak{t}_L)) + \frac{y(\mathfrak{t}_L) + \zeta\omega_n x(\mathfrak{t}_L)}{\omega_D} \sin(\omega_D(t - \mathfrak{t}_L)) \right] \\ & + \frac{1}{m\omega_D} \int_0^{(t-\mathfrak{t}_L)} (pp(s) - m e?(s)) e^{-\zeta\omega_n((t-\mathfrak{t}_L)-s)} \sin(\omega_D((t - \mathfrak{t}_L) - s)) ds \end{aligned} \quad (5)$$

$$\begin{aligned} y(t - \mathfrak{t}_L) = \mathcal{D}x(t - \mathfrak{t}_L) = & e^{-\zeta\omega_n(t-\mathfrak{t}_L)} \left[y(\mathfrak{t}_L) \cos(\omega_D(t - \mathfrak{t}_L)) - \frac{\omega_n}{\omega_D} (\zeta y(\mathfrak{t}_L) + \omega_n x(\mathfrak{t}_L)) \sin(\omega_D(t - \mathfrak{t}_L)) \right] \\ & + \frac{1}{m} \int_0^{(t-\mathfrak{t}_L)} (pp(s) - m e?(s)) \times \\ & e^{-\zeta\omega_n((t-\mathfrak{t}_L)-s)} \left(\cos(\omega_D((t - \mathfrak{t}_L) - s)) - \frac{\zeta\omega_n}{\omega_D} \sin(\omega_D((t - \mathfrak{t}_L) - s)) \right) ds \end{aligned} \quad (6)$$

The idea now is to tailor the various parameters of the model in such a way that we can prove that the form we choose for pp lets us derive the desired bound $|x| \leq X_B$.

To conform to engineering practice for this class of systems, the form we choose for pp will consist of pulses, as suggested earlier. Pulses have a large value for a short period, and are zero the rest of the time. As clearly explained in [19], if the support of a pulse is small, its precise shape has little effect on the dynamics, and only its overall impulse (i.e. integral) matters. Thus the natural temptation is to idealise the pulse into a ‘delta function’, which has zero duration but nonzero integral. Although no engineering equipment implements a delta function pulse, the idealisation simplifies calculations, and so we will pursue it here, since the deviation from a realistic pulse will be small. Technically, the idealisation also allows us to illustrate how delta functions can be handled in HEB.⁶

Tacitly, we can identify the time period T_P in Fig. 7 with the interval between pulses. Suppose then that one of these idealised delta pulses has just occurred. In the ensuing interval, the form of pp will be zero, so the pp terms can be removed from (5) and (6). Assuming that we know $x(t_L)$ and $y(t_L)$, we thus calculate the behaviour of x and y in the ensuing interval. Demanding that this remains within safe limits imposes constraints on $x(t_L)$ and $y(t_L)$, which it was the obligation of the immediately preceding pulse to have ensured. Analogously, it is the obligation of the next pulse to ensure equally safe conditions for the next interval. And so on.

Thus, we are interested in estimating the behaviour of the x and y variables during a transition of the *MONITOR* event. To this end, we argue as follows. Having tacitly arranged that the pulses occur at the transitions specified by the *MoSkip* event, and thus that pp is zero during a *MONITOR* transition, we note that the period of the building’s vibrations during an earthquake, which is typically of the order of a second or two and is captured in the constants ω_n and ζ , will be much longer than the response time of the active protection system, i.e. will be much longer than T_P . Therefore, the domain of integration in (5) and (6) will always be much shorter than a half cycle of the trigonometric terms, as a consequence of which the combined exponential and trigonometric terms will always be positive throughout the domain of integration. In such a case, the extremal values of the integral will arise when the modulating factor $e^?$ takes its own extremal values. These are just the constant values $e \pm e_B$ (the sign to be chosen depending on which one favours the argument we wish to make). Substituting these (and keeping both signs in case of future need) reduces the integrals to an analytically solvable form, which is readily evaluated [23, 26]. For a duration T_P we get:

$$x(T_P + t_L) = e^{-\zeta \omega_n T_P} \left[x(t_L) \cos(\omega_D T_P) + \frac{1}{\sqrt{1 - \zeta^2}} \left(\frac{1}{\omega_n} y(t_L) + \zeta x(t_L) \right) \sin(\omega_D T_P) \right]$$

⁶The issue is not a trivial one. HEB semantics is defined in terms of piecewise absolutely continuous functions [11]. But a delta function is not piecewise absolutely continuous, because, to be precise, it is not a function at all.

$$\begin{aligned}
& - (e \pm e_B) \frac{1}{\omega_n^2} \left[1 - e^{-\zeta \omega_n T_P} \left(\cos(\omega_D T_P) + \frac{\zeta}{\sqrt{1 - \zeta^2}} \sin(\omega_D T_P) \right) \right] \quad (7) \\
y(T_P + \mathfrak{t}_L) &= \mathcal{D} x(T_P + \mathfrak{t}_L) = \\
& e^{-\zeta \omega_n T_P} \left[y(\mathfrak{t}_L) \cos(\omega_D T_P) - \frac{1}{\sqrt{1 - \zeta^2}} \left(\omega_n x(\mathfrak{t}_L) + \zeta y(\mathfrak{t}_L) \right) \sin(\omega_D T_P) \right] \\
& - (e \pm e_B) \frac{1}{\omega_D} e^{-\zeta \omega_n T_P} \sin(\omega_D T_P) \quad (8)
\end{aligned}$$

Next we observe that the impulsive force that the active protection system applies during a pulse will not significantly change x but will only have a significant impact on y . Thus, assuming the system only reacts when $|x|$ is close to its permitted maximum value (specified by an appropriately chosen threshold value X_{th}), we infer that the following statements:

$$\text{IF } 0 < X_{th} \leq x(\mathfrak{t}_L) \leq X_B \text{ THEN ENSURE } x(T_P + \mathfrak{t}_L) \leq X_B \text{ FI} \quad \text{and} \quad (9)$$

$$\text{IF } 0 > -X_{th} \geq x(\mathfrak{t}_L) \geq -X_B \text{ THEN ENSURE } x(T_P + \mathfrak{t}_L) \geq -X_B \text{ FI} \quad (10)$$

express a policy for ensuring that the invariant $|x| \leq X_B$ is maintained throughout the dynamics of the system. These allow us to focus predominantly on Eq. (7), using (8) only occasionally.

We note that for the typical scenario of interest, $\zeta \lesssim 0.1$, so that $\sqrt{1 \pm \zeta^2} \cong 1$. From this we deduce that $\omega_n \cong \omega_D$, so we call both of them ω henceforth. Bearing the implications of such system parameters in mind, we embark on a process of simplifying (7) and (8). The observations just made lead to:

$$\begin{aligned}
x(T_P + \mathfrak{t}_L) &= \\
& e^{-\zeta \omega T_P} \left[x(\mathfrak{t}_L) \cos(\omega T_P) + \left(\frac{1}{\omega} y(\mathfrak{t}_L) + \zeta x(\mathfrak{t}_L) \right) \sin(\omega T_P) \right] \\
& - (e \pm e_B) \frac{1}{\omega^2} \left[1 - e^{-\zeta \omega T_P} \left(\cos(\omega T_P) + \zeta \sin(\omega T_P) \right) \right] \quad (11)
\end{aligned}$$

$$\begin{aligned}
y(T_P + \mathfrak{t}_L) &= \\
& e^{-\zeta \omega T_P} \left[y(\mathfrak{t}_L) \cos(\omega T_P) - \left(\omega x(\mathfrak{t}_L) + \zeta y(\mathfrak{t}_L) \right) \sin(\omega T_P) \right] \\
& - (e \pm e_B) \frac{1}{\omega} e^{-\zeta \omega T_P} \sin(\omega T_P) \quad (12)
\end{aligned}$$

Also, to ensure that the system is responsive enough to adequately dampen large oscillations coming from an earthquake, it should be prepared to respond at least 20 times per building oscillation, making $\omega T_P \cong 0.05$, and making $\zeta \omega T_P \cong 0.005$. This allows us to further simplify (11) and (12), keeping low order terms only. We work to second order in ωT_P and regard $\zeta \approx \omega T_P$. This leads to the discarding of contributions $[(1/2)\zeta^2 \omega^2 T_P^2] (y(\mathfrak{t}_L) T_P)$ to $x(T_P + \mathfrak{t}_L)$ and of $\zeta^2 \omega^2 T_P^2 y(\mathfrak{t}_L) +$

$\zeta \omega^2 T_P^2 (x(\mathfrak{t}_L) \omega) - (e \pm e_B) T_P ((1/2) \zeta^2 \omega^2 T_P^2)$ to $y(T_P + \mathfrak{t}_L)$ —these will certainly be negligible if we consider that real systems are noisy. In this way we get:

$$x(T_P + \mathfrak{t}_L) = \left[x(\mathfrak{t}_L) \left(1 - \frac{\omega^2 T_P^2}{2} \right) + y(\mathfrak{t}_L) T_P \left(1 - \zeta \omega T_P \right) \right] - \frac{(e \pm e_B) T_P^2}{2} \quad (13)$$

$$y(T_P + \mathfrak{t}_L) = \left[y(\mathfrak{t}_L) \left(1 - 2 \zeta \omega T_P \right) - \omega x(\mathfrak{t}_L) \omega T_P \right] - (e \pm e_B) T_P \left(1 - \zeta \omega T_P \right) \quad (14)$$

These formulae exhibit characteristics that we would expect. Thus, the leading contribution to $x(T_P + \mathfrak{t}_L)$ is $x(\mathfrak{t}_L) + y(\mathfrak{t}_L) T_P$, to which are added smaller corrections, while the leading contribution to $y(T_P + \mathfrak{t}_L)$ is $y(\mathfrak{t}_L)$ itself, modified by smaller corrections. The relative constancy of the velocity y over an interval T_P confirms that our proposed strategy, of imposing a pulse which discontinuously alters $y(\mathfrak{t}_L)$, will be the dominant effect on the displacement variable x during the interval. We also see that the earthquake acceleration, which contributes the $(e \pm e_B)$ terms, is not very significant unless it is violent enough to be comparable to the time period T_P or its square.

In principle (13) and (14) give us enough to design the protection system. At the end of each T_P interval, we examine $x(\mathfrak{t}_L)$ and $y(\mathfrak{t}_L)$, we calculate $x(T_P + \mathfrak{t}_L)$ according to (13), and if the answer exceeds X_B , we apply a pulse to change $y(\mathfrak{t}_L)$ to a new value $\bar{y}(\mathfrak{t}_L)$ for which a recalculated $\bar{x}(T_P + \mathfrak{t}_L)$ does not exceed X_B . However, we wish to do a bit better. We would like to identify a safe region, given by a threshold value X_{th} , such that if $x(\mathfrak{t}_L) \leq X_{th}$, no further action is needed. To identify X_{th} , we need an upper bound for $y(\mathfrak{t}_L)$ so that we can estimate how much ‘help’ the velocity could give to x during an interval. We argue as follows.

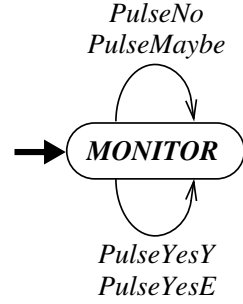
We note that starting from a stationary state, neglecting the lower order corrections (including the contribution from $x(\mathfrak{t}_L)$ whose coefficient is small), and considering the strongest earthquake the system is designed to cope with, E_B , each interval can add at most $E_B T_P$ to y . So after N intervals, y is at most $N E_B T_P$. Turning to x , an interval can similarly add at most $E_B T_P^2/2$ from the last term of (13), and after N intervals, $(1 + 2 \dots N) E_B T_P$ is added from the velocity term, giving a total, after N intervals, of $E_B T_P^2(N^2 + 2N)/2$. This must not exceed X_B , which leads to⁷:

$$N \approx \left\lceil \sqrt{2X_B/E_B T_P^2} + 1 \right\rceil \quad (15)$$

The threshold value X_{th} must be small enough that the largest possible single increment of x cannot exceed $X_B - X_{th}$. From (13), using the maximal velocity derived earlier, we get:

⁷In deriving this, we dropped a term -1 from the RHS of (15).

Fig. 8 A transition diagram for the second refinement of the HEB earthquake damage prevention model



$$X_{th} \leq X_B - \left(E_B T_P^2 \sqrt{2X_B/E_B T_P^2 + 1} + \frac{E_B T_P^2}{2} \right) \quad (16)$$

For (16) to be reasonable, its RHS must be positive, which leads to the consistency condition $2X_B \geq E_B T_P^2$. This is sensible, since if not (and referring to (13)), a single cold start interval could overreach X_B , and the threshold idea would not make sense. (The same condition is also necessary for (15) to yield a positive integer, when the discarded -1 is reinstated.)

From the account above, it is clear that if $|x(t_L)| \leq X_{th}$ at the start of an interval, then the system need do nothing. This will be the case most of the time in reality, since the only vibrations sensed will be from normal everyday activity in the building and its surroundings. However, if $|x(t_L)| > X_{th}$, then the more detailed calculation in (13) will be needed, in case there is a risk of exceeding the bound X_B .

These observations underpin the next model in our HEB development, whose transition diagram is in Fig. 8, and the text of which is in Fig. 9. In this model, the *MONITOR* event no longer has a *COMPLY* clause stipulating the behaviour of the system via an implicitly chosen *pp* function. In accordance with our discussion, the externally imposed force p is zero during *MONITOR*. The job of ensuring that the invariant $|x| \leq X_B$ is maintained becomes the responsibility of delta pulses that jolt the system into acceptable behaviour when necessary.

Here we hit a technical snag, in that delta functions do not exist in the semantics of HEB (see footnote 6). Rather than express the needed delta functions directly, we use their time integrals, which are discontinuous functions, which *do* exist in the semantics of HEB, and are typically implemented using mode events. The burden of implementing the pulses thus falls to refinements of the earlier *MoSkip* event, which implement the imposition of the needed delta functions onto the acceleration $\mathcal{D}\mathcal{D}x = \mathcal{D}y$, by instead imposing discontinuities on its integral y .

Accordingly, when time is a multiple of T_P , if $|x| < X_{th}$, then event *PulseNo* executes, and just resets the clock. But if $|x| \geq X_{th}$ then we need a more complex calculation, analogous to Eq. (13). If this reveals that the projected future $|x|$ value will nevertheless still be below X_B , then the action is the same, expressed in event *PulseMaybe*.

<pre> MACHINE ActConMch_2 REFINES ActConMch_1 CLOCK clk_pls PLIANT x,y,p INVARIANTS x,y,p ∈ ℝ, ℝ, ℝ x ≤ X_B EVENTS INITIALISATION REFINES INITIALISATION STATUS ordinary BEGIN clk_pls := 0 x,y,p := 0,0,0 END PulseNo REFINES MoSkip STATUS ordinary WHEN clk_pls = T_P ∧ x < X_{th} THEN clk_pls := 0 END PulseMaybe REFINES MoSkip STATUS ordinary ANY e? WHERE clk_pls = T_P ∧ x ≥ X_{th} ∧ e? ∈ ℝ ∧ e? ≤ E_B ∧ x(1 - ω²T_P²/2) + yT_P(1 - ζωT_P) - e?T_P²/2 ≤ X_B THEN clk_pls := 0 END MONITOR REFINES MONITOR STATUS pliant ANY e,e? WHERE e ∈ ℝ ∧ CONST(e) ∧ e? ∈ ℝ ∧ e? ≤ E_B ∧ e - e? ≤ e_B WITH pp = 0 SOLVE p := 0 Dx = y Dy = -$\frac{c}{m}y - \frac{k}{m}x - e?$ END </pre>	<pre> PulseYesY REFINES MoSkip STATUS ordinary ANY Δx,w,e? WHERE clk_pls = T_P ∧ x ≥ X_{th} ∧ e? ∈ ℝ ∧ e? ≤ E_B ∧ x(1 - ω²T_P²/2) + yT_P(1 - ζωT_P) - e?T_P²/2 - X_B = w ∧ w > 0 ∧ Δx = w + (X_B - x) ∧ [(sign(y) = sign(-e?) ∧ yT_P(1 - ζωT_P) ≥ Δx/2) ∨ (sign(y) ≠ sign(-e?) ∧ yT_P(1 - ζωT_P) ≥ Δx)] THEN clk_pls := 0 y := -y END PulseYesE REFINES MoSkip STATUS ordinary ANY Δx,w,e? WHERE clk_pls = T_P ∧ x ≥ X_{th} ∧ e? ∈ ℝ ∧ e? ≤ E_B ∧ x(1 - ω²T_P²/2) + yT_P(1 - ζωT_P) - e?T_P²/2 - X_B = w ∧ w > 0 ∧ Δx = w + (X_B - x) ∧ [(sign(y) = sign(-e?) ∧ -e?T_P²/2 ≥ Δx/2) ∨ (sign(y) ≠ sign(-e?) ∧ -e?T_P²/2 ≥ Δx)] THEN clk_pls := 0 y := e?T_P/2(1 - ζωT_P) END END </pre>
---	---

Fig. 9 Second refinement of the system

However, if the calculation reveals that without intervention X_B will be breached, then the system must intervene to prevent it. This is captured in mode events *PulseYesY* and *PulseYesE* and involves a case analysis as follows.

Let us call Δx the difference between the projected future $|x|$ value and the before-value of $|x|$ in these events, as in the two events' guards. Then if Δx turns out positive, it can only be because either the y term or the $-e?$ term of the projected future $|x|$ value, or both, is/are driving $|x|$ too high. At least one of these terms has a value whose sign is the same as that of the before-value of x in the two events, else both terms would drive $|x|$ smaller, contradicting the breaching of X_B . N.B. We assume that the threshold is big enough that above threshold, a single interval cannot cause x to change sign, and thus cannot cause $|x|$ to increase even in cases in which the rate of change of x changes sign.

Suppose then that both terms have values whose sign agrees with that of the value of x . Then one of them has a value which is at least $\Delta x/2$ since they act additively and their sum is Δx . In this case it is sufficient to invert the sign of the larger contribution to ensure that their net effect diminishes $|x|$. So we either flip y , or flip a suitably rescaled $e?$. This covers one of the two cases in each of *PulseYesY* and *PulseYesE*.

Suppose alternatively that only one of the terms has a value whose sign agrees with that of the value of x . Then the magnitude of that term must exceed Δx , since they act subtractively and the difference of their magnitudes is still Δx .⁸ In this case it is sufficient to invert the sign of this larger contribution to ensure their net effect diminishes $|x|$. This covers the remaining two cases in *PulseYesY* and *PulseYesE*.

8 On HEB Refinement

At this point we reflect on the refinement just done. A first point notes that during normal Event-B refinement [3], the behaviour of an event is typically restricted, making it more deterministic. In our case, we have taken this to an extreme, by effectively abandoning external control of the behaviour of the dynamical variables x and y via p during *MONITOR*, and have delegated this duty instead to the *PulseXX* events. So the *PulseXX* events are new in the model of Fig. 9, and define new behaviour for y (and potentially for x too, if it were needed). This is against the rules of Event-B refinement, since new behaviour for variables should be introduced at the same time as the variables themselves (being made more deterministic subsequently)—whereas we introduced x and y during the previous refinement.

We partly mitigated this by making the *PulseXX* events refine the earlier *MoSkip* events, introduced during the previous refinement stage, and giving the *MoSkip* events the status 'anticipating'. This status allows an event, newly introduced during a refinement step, and which would normally be required to strictly decrease a relevant

⁸Mathematically, it is possible for both terms to have magnitude bigger than Δx , unless we take into account relevant upper bounds etc. and show that it is impossible. We will just assume that there is no such possibility in our problem space.

variant function (to ensure the convergence of the new behaviour), to *not strictly* decrease the variant then, postponing this obligation till later.⁹ Since we included no variants in our development, we discharged this duty trivially. The introduction of *MoSkip* and variable y at the same time thus not only allowed fresh choice of e in successive iterations of *MONITOR* but allowed the manipulation of y in refinements of *MoSkip*.

Unfortunately though, by not mentioning y at all, *MoSkip* by default specifies that y does not change during *MoSkip* transitions, while the *PulseYes* events refining it specify nontrivial changes to y . It is tempting to think that this is still a refinement, since the only invariant concerning y is $y \in \mathbb{R}$, the weakest possible invariant. However, when the same variable exists in a machine and in its refinement, there is an implicit equality invariant between the abstract and concrete versions of the variable—otherwise writing more conventional refinements would become intolerably verbose. In this regard, ‘no change’ in *MoSkip* is incompatible with ‘nontrivial update’ in *PulseYes*, and our refinement isn’t quite legal after all. This shows that ‘refining to a delta function’ is not ideally handled in HEB. The only way to make the development unimpeachable according to the rules of Event-B is to introduce the variable y and the nontrivial mode event behaviour at the same time. But this is less desirable from our point of view, as it forces the choice of control strategy without permitting consideration of alternatives, and flies in the face of the objectives of a refinement driven development strategy which aims at introducing detail into designs in stages.

A second point concerns the arguments we employed in the preceding pages. Our reasoning started out being quite watertight mathematically, but rather quickly, we started to introduce simplifications which were perfectly justifiable on engineering grounds, but which would not pass the unblinking scrutiny of formal proof. Two centuries or more of rigorous mathematical analysis have, in principle, developed techniques, using which, such a shortcoming could be overcome, but the amount of work involved would be considerable, and would quickly surpass the small amount of added assurance that could be gained. The formal development field, in its somewhat strenuous avoidance of engagement with continuous mathematics hitherto, has not really developed a cost effective approach to dealing with this issue.

A third point concerns the extent to which the model of Fig. 9 can actually be proved correct using a *per event* proving strategy as embodied in the semantics and verification architecture of Event-B. This is by contrast with the arguments in preceding pages, which focused on the application structure and employed whatever observations seemed useful at the time, without regard to how the various points were to be structured into an overall correctness argument. Here, the news, though not perfect, is better.

We note that the *MONITOR* event, as written in Fig. 9, cannot by itself be correct according to the normal ‘preserving the invariant’ notion of event correctness, since

⁹The formal presentation of HEB [11] does not mention the anticipating status, since that is somewhat outside the main concerns there. But there is no reason to forbid it since it concerns purely structural matters.

it demands no restrictions on $x(\mathfrak{t}_L)$ and $y(\mathfrak{t}_L)$. Without prior knowledge about these, the ODE system can easily breach the $x \leq X_B$ bound during a T_P interval. Of course, we rely on the *PulseYes* events to ensure appropriate $x(\mathfrak{t}_L)$ and $y(\mathfrak{t}_L)$ values for the subsequent *MONITOR* event, but the *MONITOR* event correctness proof obligations know nothing of this. However, in HEB, we also have ‘well-formedness’ proof obligations, that police the handover between mode and pliant events. These can check that after any of the *PulseXX* events, the values of x and y are appropriate. In particular, they check that after the *PulseXX* events the guard for at least one pliant event is true. Since we have designed the *PulseXX* events to ensure exactly what is required here, the trivial guard of the *MONITOR* event of Fig. 9 could, in fact, be strengthened to demand a suitably stringent constraint on $x(\mathfrak{t}_L)$ and $y(\mathfrak{t}_L)$, from which, ‘preserving the invariant’ would become possible. So, although we did not get diverted by this detail earlier, a solution entirely within the rules is available.

9 Sensors, Actuators, Sampling, Quantization, Decomposition

The next model in our development tackles a number of issues that add low level complexity. Following the structure of [20, 21], we introduce a sensor and an actuator into the system. Elements like these bring various kinds of imprecision to the development. Thus, they typically act at discrete moments of time—this brings temporal imprecision. Their inputs and outputs typically have finite ranges, and are quantized—this brings imprecision of magnitude. The impact of these sources of imprecision is similar from a formal point of view, and describing these phenomena precisely, generates complexity in the textual description of the system.

Moreover, a model close to the architectural structure of [20, 21] would place the architecturally distinct components of the system in separate constructs. To create such a model requires the decomposition of a monolithic version into smaller pieces, a process which, if done with precision, generates both textual complexity and a lot of repetition of the model text.

To minimise verbosity, our strategy will therefore be as follows. Viewing the model of Fig. 9 as being at level 2, the level 2 model is conceptually developed into an unstated, but still monolithic model, incorporating the features mentioned above, at level 3 (with machine *ActConMch_3* say). This is then decomposed into a multimachine project at level 4, exhibiting the desired architectural structure. The level 4 model is presented in Figs. 11, 12, 13 and 14, and described below. We comment more extensively on the level 4 model later on.

In Fig. 10 we have a depiction of the various HEB machines of the distributed concurrent HEB model that results from the process just sketched. Figures 11, 12, 13 and 14 contain the text of the resulting model. We start with the PROJECT *ActCon_4_Prj* file in Fig. 11, which describes the overall structure. The DECOMPOSES *ActCon_3_Prj* line refers to the fictitious level 3 system, of which more

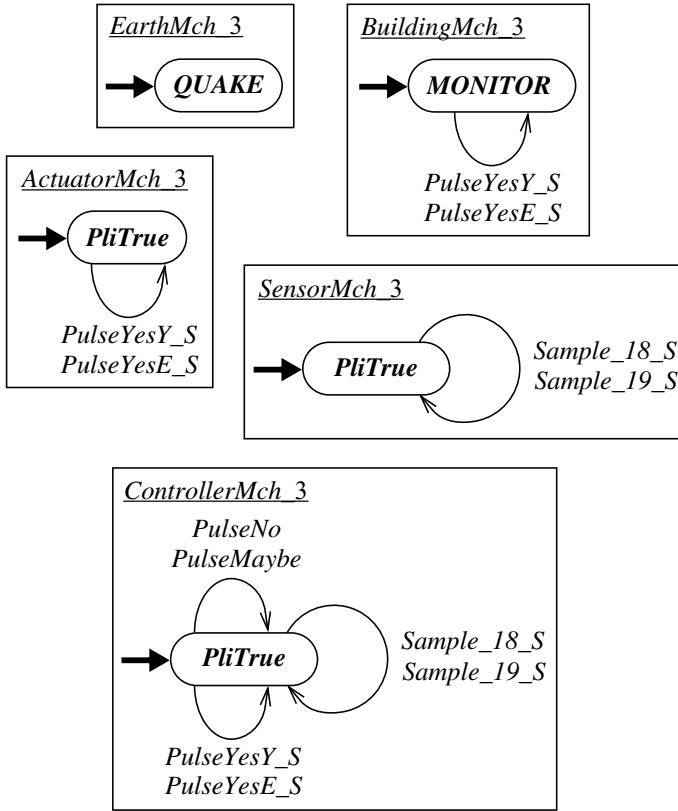


Fig. 10 A family of transition diagrams for the HEB machines of a distributed concurrent version of the active earthquake damage prevention system

later. The main job of the PROJECT file is to name the constituent machines and interfaces, and to define needed synchronisations between the mode events of the different machines. Thus there are machines for the earth, the building, the actuator, the sensor, and the controller. The PROJECT file also names the INTERFACE *ActCon_4_IF* file. This declares any variables that are shared between more than one machine, their initialisations, and, most importantly, any invariants that mention any of these variables. (The latter point can place stringent restrictions on how variables are partitioned into different interfaces and machines.) The final responsibility of the PROJECT file is to declare the mode event synchronisations. Thus SYNCHronisation *Sample18* specifies that mode event *Sample_18_S* in machine *SensorMch_4* and mode event *Sample_18_S* in machine *ControllerMch_4* must be executed simultaneously. This means that they can only execute if all the guard conditions in all the events of the synchronisation are true. The same remarks apply to the other synchronisations declared in the project file.

<pre> PROJECT ActCon_4_Prj DECOMPOSES ActCon_3_Prj MACHINE EarthMch_4 MACHINE BuildingMch_4 MACHINE SensorMch_4 MACHINE ControllerMch_4 MACHINE ActuatorMch_4 INTERFACE ActCon_4_IF SYNCH(Sample18) SensorMch_4.Sample_18_S ControllerMch_4.Sample_18_S END SYNCH(Sample19) SensorMch_4.Sample_19_S ControllerMch_4.Sample_19_S END SYNCH(PulseYesY) ActuatorMch_4.PulseYesY_S BuildingMch_4.PulseYesY_S ControllerMch_4.PulseYesY_S END SYNCH(PulseYesE) ActuatorMch_4.PulseYesE_S BuildingMch_4.PulseYesE_S ControllerMch_4.PulseYesE_S END END </pre>	<pre> INTERFACE ActCon_4_IF PLIANT xx,yy,ee INVARIANTS xx,yy,ee ∈ ℝ,ℝ,ℝ xx ≤ X_B INITIALISATION xx,yy,ee := 0,0,0 END </pre>
--	--

Fig. 11 The PROJECT and INTERACE files of the further developed and decomposed system

We turn to the machines, pictured in Fig. 10. In outline, machine *EarthMch_4* is responsible for producing the earthquake acceleration, which comes from the input $e?$, as in previous models. This is simply captured during the pliant event *QUAKE* and is recorded in the shared pliant variable ee , declared in the interface (which the *EarthMch_4* machine CONNECTS to). We can see that the *QUAKE* event comes from decomposing the earlier *MONITOR* pliant event, and we will see the remnants of the *MONITOR* event elsewhere soon. Since any HEB machine must have at least one pliant event to describe what happens over the course of time, but need not contain any other event, and since *QUAKE* addresses that requirement, there are no other events in *EarthMch_4*.

The shared variable ee is accessed by machine *BuildingMch_4*. This contains the remainder of the earlier *MONITOR* pliant event, namely the ODE system defining the building's response, which uses ee . It also contains the business end of the *PulseYesY_S* and *PulseYesE_S* mode events, which take their inputs (which are received from the actuator using input $ys?$) and discontinuously impose the received values on the velocity variable yy .

We come to the *SensorMch_4* and *ActuatorMch_4* machines. Their behaviour is essentially discrete, so to satisfy the requirement for having a pliant event, both



Fig. 12 Machines for earth, building, sensor and actuator

machines have a default COMPLY *INVARIANTS* pliant event, named, as is typically the case, *PliTTrue*. In fact, since all the pliant variables are handled by other machines, there is nothing for these *PliTTrue* events to do, and that is part of the semantics of ‘COMPLY *INVARIANTS*’ in HEB.

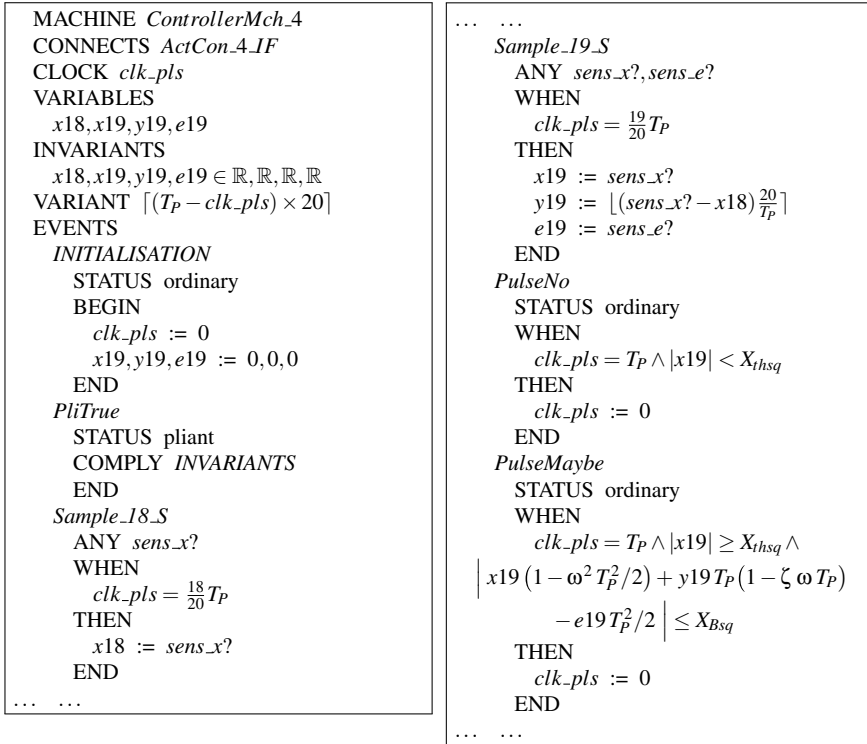


Fig. 13 The controller machine, first part

The job of the *SensorMch_4* machine is to sample the physical values required by a idealised implementation of the system. The values are required at pulse issuing time, but to allow time for computation, as in [20, 21], they are collected a little earlier. The position and earth acceleration values, from *xx* and *ee*, are collected 19/20 of the way through a T_p interval, and are transmitted (to the controller machine) in output variable *sens_x!* and *sens_e!* An extra position value is needed for calculating a velocity estimate, so another sample of *xx* is taken 18/20 of the way through T_p . Notice that the *xx* and *ee* values are scaled (by K_{xsqs} and K_{esqs}), rounded, and then unscaled (by K_{xsqs}^{-1} and K_{esqs}^{-1}) before sending, to model the quantization process.¹⁰ The mode events that do these jobs are *Sample_18_S* and *Sample_19_S*. The ‘_S’ suffixes on these names indicate, for readability, that these are synchronised with mode events in one or more other machines, though, as we mentioned earlier, the formal definition of a project’s synchronisations are in the project file.

The same general comments work for the *ActuatorMch_4* machine. Only the velocity variable is modified in our development, so only this variable is acted on by

¹⁰In reality, a sensor would send values in its own units, and scaling would be done as part of the controller’s job, but we avoid this so as to keep the controller calculation reasonably transparent.

the actuator. The value needed is received (from the controller machine) in the *act_y?* input of synchronised events *PulseYesY_S* and *PulseYesE_S*, and after quantization via K_{ysqs} and its inverse, is transmitted (to the building machine) in the *ys!* output.¹¹ As for the sensor, there is no need for any non-trivial pliant event, so a default *PliTrue* suffices.

At the heart of the system is the *ControllerMch_4* machine. This houses the remaining functionality, and the non-trivial computation. The clock *clk_pls* is declared here, as are local variables $x18, x19, y19, e19$, the sampled values of the dynamical variables, which are not needed in any other machine. We see also that *ControllerMch_4* only requires the *PliTrue* default pliant event, since its interventions are exclusively at individual moments of time. It also contains the remaining portions of the various synchronisations we have discussed.

The *Sample_18_S* event picks up the sampled position at times $18/20 T_P$ of an interval, recording them in $x18$. The *Sample_19_S* event picks up position and acceleration samples at $19/20 T_P$ and, as well as recording these, it calculates an estimate of velocity from the position samples and records it in $y19$. The values in $x19, y19, e19$ are then ready for the pulse calculations, which would consume some time to do, but which are modelled as taking place instantaneously at the end of the interval in the various *Pulse* events.

Compared with the other events, events *Sample_18_S* and *Sample_19_S* are newly introduced in this development step. In such a case, Event-B practice asks that they strictly decreases some variant function, which is included in Fig. 13 after the invariants. It is clear that at the two occurrences of the *Sample* events in each interval, the value of the variant drops, firstly from $\lim_{\varepsilon \rightarrow 0+} \lceil (T_P - (\frac{18}{20}T_P + \varepsilon)) \times 20 \rceil = 3$ to 2, and then from 2 to 1,¹² thus strictly decreasing it, as required.

The *PulseNo*, *PulseMaybe*, and now synchronised *PulseYesY_S* and *PulseYesE_S* events, handle the needed responses to building movement, as before, except that the calculations are now done using the sampled, quantized (SQ) values rather than the ideal, instantaneous (II) ones. This inevitably leads to disagreement with the ideal calculations in the border country where different behaviour regimes meet (in our case, the border country between the do pulse and don't pulse regimes).¹³

In our case, we have to cope with the possibility that the SQ values dictate a pulse in a situation where the II values don't (which is tolerable, since it will only happen in the border country, where pulses are probable anyway), or that the SQ values don't dictate a pulse in a situation where the II values do (which is *intolerable* since it may permit the physical system to overshoot the X_B bound without the SQ model being aware of it). We must prevent the latter.

¹¹On a technical level, the building and actuator machines illustrate the pattern whereby synchronised mode events in different machines can instantaneously share values: one event uses an output variable and the others use an input variable with a complementary (CSP style) name.

¹²Note that this critically depends on insisting that intervals of pliant behaviour are left closed and right open.

¹³Henceforth, we will use SQ to refer to and to label elements and quantities relevant to the level 4 model of Figs. 11, 12, 13 and 14 (and, implicitly to its unstated level 3 precursor), and II for elements relevant to the level 2 model of Fig. 9, as needed.

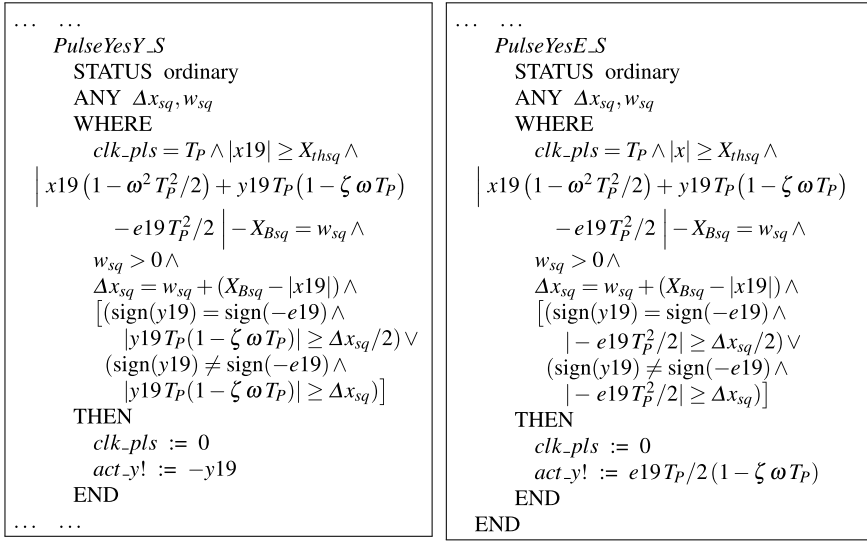


Fig. 14 The controller machine, second part

The approach we take is to conservatively adjust the constants X_{th} , X_B in the model to new values X_{thsq} , X_{Bsq} that preclude the intolerable omissions at the price of admitting more superfluous pulses.¹⁴ For more convenient discussion, we also renamed the local variables Δx , w in Fig. 14 by adding a subscript.

Our remarks indicate that whenever $PulseNo^{SQ}$ or $PulseMaybe^{SQ}$ can run, then we must be sure that $PulseNo^I$ or $PulseMaybe^I$ will also run. This implies a condition on their guards.

We take the events individually, starting with $PulseNo^{SQ}$ and $PulseNo^I$. It is clear that the latter is enabled whenever the former is, provided that $|x19| < X_{thsq} \Rightarrow |xx| < X_{th}$ holds. Of course, $x19$ and xx refer to values at different times, but recalling that X_{th} was derived by estimating the maximum achievable displacement over a whole interval in (16), one twentieth of the same argument will cover the difference between $x19$ and xx . So our implication will hold, provided:

$$X_{thsq} \leq X_{th} - \left(E_B T_P^2 \sqrt{2X_B/E_B T_P^2 + 1} + \frac{E_B T_P^2}{2} \right) / 20 \quad (17)$$

We see that this is a small correction to X_{th} , which, for typical parameter values, will be negligible in practice, if not in mathematics, confirming the conjecture in footnote 14.

¹⁴Speaking realistically, in a genuine earthquake scenario, noise and experimental uncertainty are likely to be such that the differences between the ideal and conservative values of the constants vanish into insignificance. But it is worth checking that the mathematics confirms this.

Turning to $PulseMaybe^{SQ}$ and $PulseMaybe^{\Pi}$ a similar argument applies. Looking at the relevant guards, we see that as well as (17), we will be able to maintain the invariant $|xx| < X_B$ provided we make an analogous correction to X_B for the purpose of the estimates made in the $PulseMaybe^{SQ}$ guard:

$$X_{BsQ} \leq X_B - \left(E_B T_P^2 \sqrt{2X_B/E_B T_P^2 + 1} + \frac{E_B T_P^2}{2} \right) / 20 \quad (18)$$

With these two cases understood, we see that the final two events are covered also. Both $PulseYesY_S^{SQ}$ and $PulseYesE_S^{SQ}$ flip the sign of the greatest contribution to the estimated increment in displacement, based on the same estimate made in $PulseMaybe^{SQ}$.

10 Refinement, Retrenchment and Other Technical Issues

In Figs. 11, 12, 13 and 14 the only structural directives are DECOMPOSES $ActCon_3_Prj$ in the project file, and the CONNECTS $ActCon_4_IF$ in the various machine files. There are a number of reasons for this. Firstly, we are presuming that the hard work of refining $ActConMch_2$ to incorporate the sensor, actuator and discretization features will have been achieved in the (unstated) $ActConMch_3$ machine, the only element of the (unstated) level 3 project $ActCon_3_Prj$.¹⁵ This understood, the job of decomposing a monolithic $ActConMch_3$ machine into the components seen in Figs. 11, 12, 13 and 14 is properly covered by the cited directives. Before continuing, we briefly comment on this by envisaging how Figs. 11, 12, 13 and 14 might be reassembled into a single construct.

Let us start with the two $Sample_18_S$ events, shared between $SensorMch_4$ and $ControllerMch_4$, and executed synchronously. In a monolithic $ActConMch_3$ (which would take on the duties of both machines), there would be a single $Sample_18$ event, with guard $clk_pls = \frac{18}{20}T_P$ and action $x18 := K_{xsqs}^{-1} [K_{xsqs} \ xx]$. There is no communication, since all the variables are accessible to the one machine. $Sample_19$ follows a similar pattern, with two variables assigned. Thus is the sensor machine's functionality absorbed into one encompassing machine.

The actuator is dealt with similarly, except that the building is involved; the functionality of the building is also absorbed into the single encompassing machine, rather as was the case in the level 2 and earlier models. The earth machine is similarly absorbed, eliminating the need for the shared variable ee . This account illustrates, in reverse, how the distributed model of Figs. 11, 12, 13 and 14 is arrived at, presuming the preexistence of the monolithic version. Note that it is a deliberate design objective of the multimachine HEB formalism that the monolithic and distributed versions should be, in all important aspects, semantically indistinguishable; see [12].

¹⁵We can also regard all the previous models as each being in its own single machine project.

Thus, the *ActConMch_3* is relatively easily imagined, avoiding some verbosity. Less easy is its relationship to the level 2 machine *ActConMch_2*—the discussion in Sect. 8, on implicit equality invariants, flags up that the introduction of imprecision via sampling and quantization may not be unproblematic regarding refinement methodology.

The immediate problem was avoided by renaming variables x, y to xx, yy in the level 4 model. But this raises the question of what the relationship between x, y and xx, yy ought to be. It is a truism in formal development that the stronger the invariants you write, the harder the work to prove that they are maintained, but the stronger the assurance that is gained thereby. And conversely. We might thus ease our task by omitting completely any non-trivial relationship between x, y and xx, yy . But this will not do since we still have the level 2 invariant $x \leq X_B$ to establish, which is rendered impossible in the level 4 model without some coupling between x, y and xx, yy .

The obvious relationship to consider is some sort of accuracy bound relating x and xx , and y and yy . Since the damping factor ζ is positive, the dynamics is asymptotically stable, so we can expect the dynamics to be contracting¹⁶ (although a refinement relationship based on this still often requires appropriate conditions on the constants of the system [35]). To see the contracting nature of our dynamics we first need to rewrite (13) and (14) in terms of dimensionally comparable quantities, for example, in terms of $\tilde{x} \equiv x$ and $\tilde{y} \equiv y/\omega$. When this is done, (13) and (14), viewed as a matrix operating on differences in pairs of values of $\tilde{x}, \tilde{y}$, has entries $(\delta_{ij} + (-1)^{i \geq j} \varepsilon_{ij})$, where δ_{ij} is the identity, and the ε_{ij} are small and positive, from which the contracting nature of the transformation can be inferred.

With this, we can claim that a single execution of *MONITOR* in each of the II and SQ systems will maintain a joint invariant of the form $\|(x, y) - (xx, yy)\|_1 \leq A$,¹⁷ provided it is true at the start, but it does not tell us what value we would need to choose for A for this to be true non-trivially.

The latter problem would require a global analysis which could be quite challenging. The issue is made the more difficult by the possibility mentioned before, whereby imprecision caused by conservative design in the SQ system causes the SQ system to express a pulse whereas the II system does not. If this happened, the $\|\cdot\|_1$ distance between the II and SQ systems would suddenly increase dramatically, even if it was well behaved previously, and it would consequently cause the $\|\cdot\|_1$ norm to function poorly as a joint invariant between II and SQ systems, posing a significant impediment to refinement as an convincing notion for relating the II and SQ systems.

A weakening of the highly demanding refinement concept is the idea of retrenchment [13, 14, 30]. In retrenchment the demand to preserve a ‘nearness’ invariant is relaxed by permitting different conditions to be specified for the before-state and after-state of a pair of transitions in the two systems being compared, and allows constants, such as A , to be declared locally per transition instance, rather than globally, as in a refinement relation. This formulation also permits the two systems to

¹⁶In a contracting dynamics, nearby points are driven closer by the dynamics.

¹⁷The $\tilde{\cdot}$ refers to an L_1 norm on the (instantaneous values of the) tilde variables.

part company during exceptional circumstances. It works well enough if the two systems quickly recover ‘nearness’, or if the models cease to be relevant at all after the exception.

In our case, the ‘exceptional’ regime, requiring pulses, is precisely the *raison d’être* of the whole protection system, and it is in this regime (rather than the normal, stable regime when there is no earthquake) in which the behaviours of the II and SQ systems are the most unruly. And although retrenchment, as described in [13, 14, 30], addresses the onset of unruly behaviour quite well, it does not really engage with particular properties of extended periods of unruly behaviour, as we would ideally like in our application.

A further complication of the scenario where the SQ system pulses and the II system does not, is that different events in the two systems are involved in these behaviours (*PulseNo* and *PulseMaybe* in II and *PulseYesY_S* and *PulseYesE_S* in SQ). Retrenchment and refinement, as usually defined, assume a static (and partial if needed) bijection between operations/events in the two models being compared. This does not cope with the scenario just mentioned, in which overlapping pairs of (names of) events may need to be related at different times.

Thus our reticence in writing down an explicit level 3 system (with its obligation to make clear its relationship to the level 2 system) is further explained by the absence of a suitable species of formal relationship that could be used for the purpose. Without getting embroiled in too many further details, the present case study provides a fertile stimulus for developing a richer formulation of retrenchment and refinement capable of coping with the wealth of phenomena it exhibits.

11 Experiments and Simulations

In this section, we compare the expectations raised by the preceding analytical work, with the outputs of well established conventional earthquake protection design approaches, based on numerical simulation.

Our simulations were performed over a time interval from 0 to $\mathcal{T}_{\text{MAX}}$, using a control strategy that, as suggested by the analytical work, is defined by a pulse interval T_P , allowable relative displacement X_B , and an additional ground acceleration variability term e_B . At the start of a pulse interval, the simulation chooses whether to apply a pulse based on Eq. (14), predicting a value of x at the end of the interval from the expression $h_x x + h_y y + h_e (e \pm e_B)$, where:

$$h_x = 1 - \frac{\omega^2 T_P^2}{2} \quad h_y = T_P \left(1 - \zeta \omega T_P\right) \quad h_e = -\frac{T_P^2}{2} \quad (19)$$

using actual values of—or available estimates for— x , y , and e at time t .

Figure 15 shows the essence of a Python program that performs simulations using the numerical and scientific libraries **NumPy** and **SciPy**, as well as the **Matplotlib** library for plotting; the complete code is available online [8].

```

1 def simulate( $\mathcal{T}_{\text{MAX}}$ ,  $T_P$ ,  $X_B$ ,  $e_B$ ):

2     def x_future(x, y, t):
3         f = lambda s:  $h_x * x + h_y * y + h_e * (e(t) + s * e_B)$ 
4         a, b = f(1), f(-1)
5         return (a, 1) if abs(a) > abs(b) else (b, -1)

6     def y_new(x_desired, x, t, s):
7         return ( $x_{\text{desired}} - h_x * x - h_e * (e(t) + s * e_B)$ ) /  $h_y$ 

8     x, y, t =  $x_0, y_0, 0$  # initial condition
9     for i in range(int( $\mathcal{T}_{\text{MAX}}/T_P$ )):
10        xe, sign = x_future(x, y, t)
11        if abs(xe) >  $X_B$ :
12            y = y_new(signum(xe) *  $X_B$ , x, t, sign)
13        x, y, t = advance(x, y, t, (i + 1) *  $T_P$ )

```

Fig. 15 Python program for numerical simulation [8]

Function **simulate**($\mathcal{T}_{\text{MAX}}, T_P, X_B, e_B$) contains two nested functions, one to predict future values of x , and another to adjust current values of y , if needed, when a pulse is called for. In particular, function **x_future**(x, y, t) estimates x at a time $t + T_P$ in the future, returning the estimate and the sign used for the e_B term that maximizes the absolute value of the estimate—the worse case. The value returned by function $e(t)$ is the ground acceleration at time t . Function **y_new**(x, y, t) likewise uses Eq. (14), but in this case does so to find a new value of y that would, one hopes, cause $|x(t + T_P)| \leq X_B$ to be satisfied; the sign for e_B must be supplied (in this case, by the result from **x_future**).

As with the HEB model, the simulation (defined by lines 8–13) is broken up into a succession of subintervals, each with duration T_P . Between subintervals, a pulse may be applied that changes the value of y instantaneously. During a subinterval, time marches from $i T_P$ to $(i + 1) T_P$.¹⁸ Function **advance**(x, y, a, b), not shown, lets the system evolve from time a to time b , starting from the initial values $x(a)$ and $y(a)$; it returns their values at time b : $x(b)$, $y(b)$, and b . As a side effect, it builds up collections of data for plotting time histories of x and y .

With respect to numerical integration, *advance* solves the system of first order differential equations:

$$\mathcal{D}x = y \quad (20)$$

$$\mathcal{D}y = -2\zeta\omega_n y - \omega_n^2 x - e(t) \quad (21)$$

¹⁸The form *for i in range(n)* is idiomatic Python for bounded iteration from 0 to $n - 1$ (inclusive).

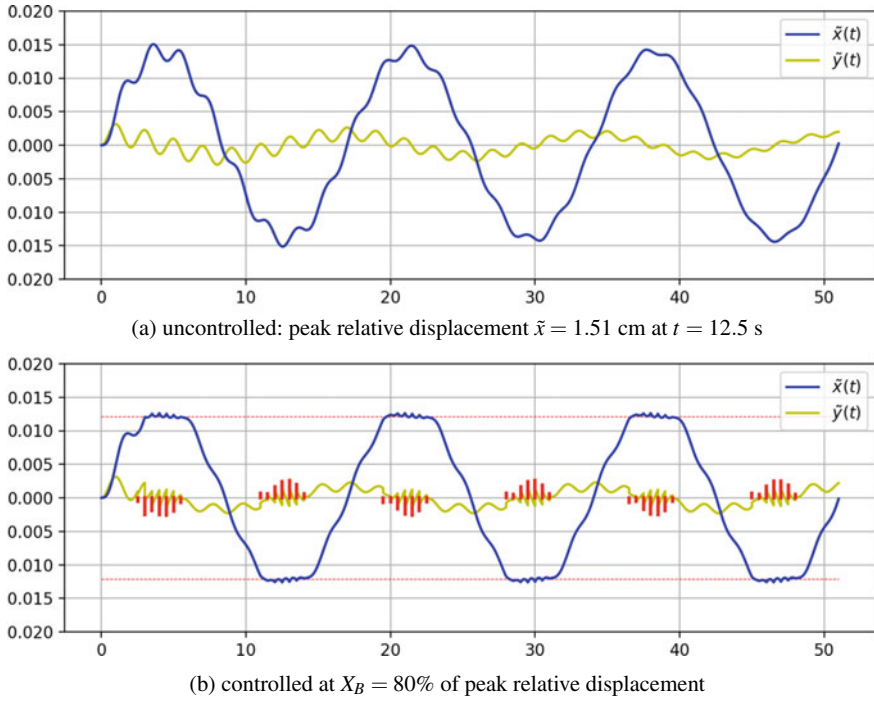


Fig. 16 Response to harmonic ground motion ($T_n = 2$ s, $\zeta = 1\%$, $Z = 1$, $\Omega = 0.37$ rad/s)

introduced in Eq. (3) and subsequently redefined in LHS (4) in terms of the variables: ζ , the viscous damping factor (dimensionless fraction of critical damping); and ω_n , the undamped circular natural frequency (in units of radians per second). It does so using `odeint`, a **SciPy** function based on the Fortran **LSODA** routine from the **ODEPACK** library, which uses an Adams predictor-corrector method (when non-stiff problems like ours are encountered). The routine determines step size automatically to ensure that error bounds are satisfied.

Harmonic ground motion. To illustrate the approach, we begin with a simple example after Prucz et al. [29] of an SDOF system, like that of Fig. 3, with a natural frequency $\omega_n = \pi$ rad/s and viscous damping factor $\zeta = 1\%$. It is subjected to harmonic ground motion $z = Z \sin \Omega t$, which has the effect of adding a reversed inertia force $-m \mathcal{D}^2 z$ to the system, with the ground acceleration given by:

$$e(t) \equiv \mathcal{D}^2 z = -\Omega^2 Z \sin \Omega t \quad (22)$$

where amplitude $Z = 1$ and frequency $\Omega = 0.37$ rad/s are given. Thus, the case is one in which the ground motion frequency is lower than the system natural frequency (i.e., $\Omega < \omega_n$). The system begins at rest, so $x_0 = y_0 = 0$.

Time histories of the uncontrolled response are shown in Fig. 16a, where the dimensionally comparable quantities $\tilde{x} \equiv x$ and $\tilde{y} \equiv y/\omega_n$ are plotted. The peak responses are:

$$\tilde{x}(12.5 \text{ s}) = -0.0151, \quad \tilde{y}(0.985 \text{ s}) = 0.00315$$

The predominant response of $\tilde{x}(t)$ is a harmonic having the same frequency as that of the ground acceleration; its period is $2\pi/\Omega$, or in this case about 17 s. As expected, when $\Omega \ll \omega_n$ there is little relative motion between the mass and the ground, and the motions are in phase: they reach their peaks at the same time. Superimposed ‘wiggles’ are (dying) transients induced at the natural frequency of the system, whose undamped natural period $T_n = 2$ s.

Pulse control can now be employed to limit the response to 80% of the peak relative displacement, which is done by setting $X_B = 0.0121$. Continuing to be consistent with Prucz et al., we set the pulse interval to be on the order of one fourth the natural period, so $T_p = T_n/4 = 0.5$ s. Specific to our approach, the additional ground acceleration variability term e_B is set to zero for the moment. Time histories of the controlled response are shown in Fig. 16b, where the pulse trains (in red) have a ‘shape’ that acts to counterbalance relative displacements, where needed, that would have occurred, so as to keep them roughly within desired limits.

Though the pulse interval here is about five times larger than we would anticipate using in practice, the example motivates the definition of a metric, the *exceedance level*, that can be used to assess the algorithm’s effectiveness as a bounded state control strategy. To quantify the exceedance level, we consider what happens at the endpoint of a T_p interval where, if $|x(t)| > X_B$, we add $|x(t)| - X_B$ to a running sum $\mathcal{S}$, and define:

$$\mathcal{E} = 10^3 \mathcal{S} / n X_B \quad (23)$$

where n is the number of pulse intervals included in sum $\mathcal{S}$. For the Prucz example, that gives an exceedance level $\mathcal{E} = 9.07$. For pulses at 20 times per natural period instead (i.e., for $T_p = 0.1$ s), we have $\mathcal{E} = 0.0250$, and when in addition e_B is raised to 0.001, the exceedance level $\mathcal{E}$ drops to zero, meaning there are no exceedances.

El Centro ground motion. As noted by Prucz et al., the aim of pulse control is to disrupt, at resonance, the ‘gradual rhythmic build-up’ of the system response. A more realistic and challenging scenario then is to subject the system to complex ground accelerations that include resonant frequencies, particularly ones near the fundamental natural frequency of a building, which typically produce the largest relative displacements and damage.

Used in the design of earthquake resistant structures, the ground accelerations recorded in El Centro, California, during the earthquake of May 18, 1940, have a peak value of 3.13 m/s^2 ($0.319g$), the first 20 s of which are shown in Fig. 17. We now apply them to the system. As before, the natural frequency $\omega_n = \pi$ rad/s, so the

undamped natural period $T_n = 2$ s, a value that might correspond to the fundamental natural period of a 20-story building. We use a viscous damping factor $\zeta = 5\%$, which is representative of a modern office building and is a value often used in design. For control, the pulse interval $T_P = 0.1$ s.

Time histories of the uncontrolled response are shown in Fig. 18a, where we again plot $\tilde{x}$ and $\tilde{y}$. The peak responses are

$$\tilde{x}(6.37 \text{ s}) = 0.137, \quad \tilde{y}(11.7 \text{ s}) = 0.199$$

which occur during a time period from about 6–13 s into the event, as the system begins oscillating near its undamped natural frequency $\omega_n = \pi$ rad/s (with a period $T_n = 2$ s).

To limit the peak displacement, we apply pulse control at 80% of that value by setting $X_B = 0.109$ (or 10.9 cm) and keep the pulse interval as before, $T_P = 0.1$ s. Time histories of the controlled response are shown in Fig. 18b, where the pulses, shown in red, effectively counterbalance relative displacements to keep them approximately within desired bounds. Limiting displacements even further, to 50 and 40% of the peak value, is likewise shown to be effective, as demonstrated by the time histories in Figs. 18c–d, respectively. To achieve the additional level of control requires that successively more energy be put into the system, with more and sometimes larger pulses, and earlier into the event.

Looking at exceedance for the three levels of controlled response (80, 50, and 40%), we have $\mathcal{E} = 0.223, 0.664$, and 0.561 , respectively, which are reduced to zero when the additional ground acceleration term, e_B , is increased to at least 0.512, 0.358, and 0.469, respectively. Additional analysis, that might lead to finding good e_B settings a priori for anticipated ground motions, is left for future work.

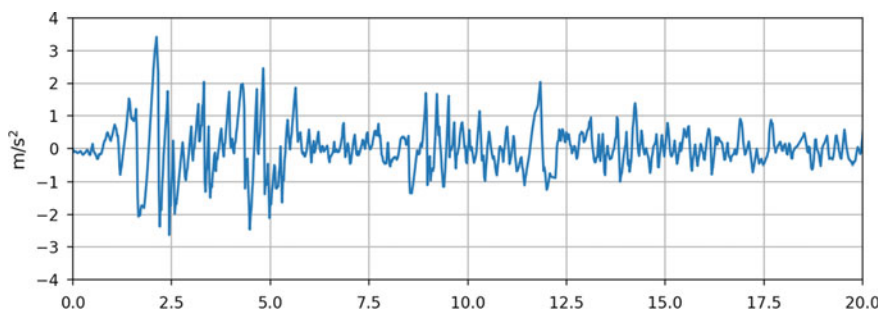


Fig. 17 North-south component of the ground motion recorded at a site in El Centro, California, during the Imperial Valley earthquake of May 18, 1940 (showing first 20 s of the event)

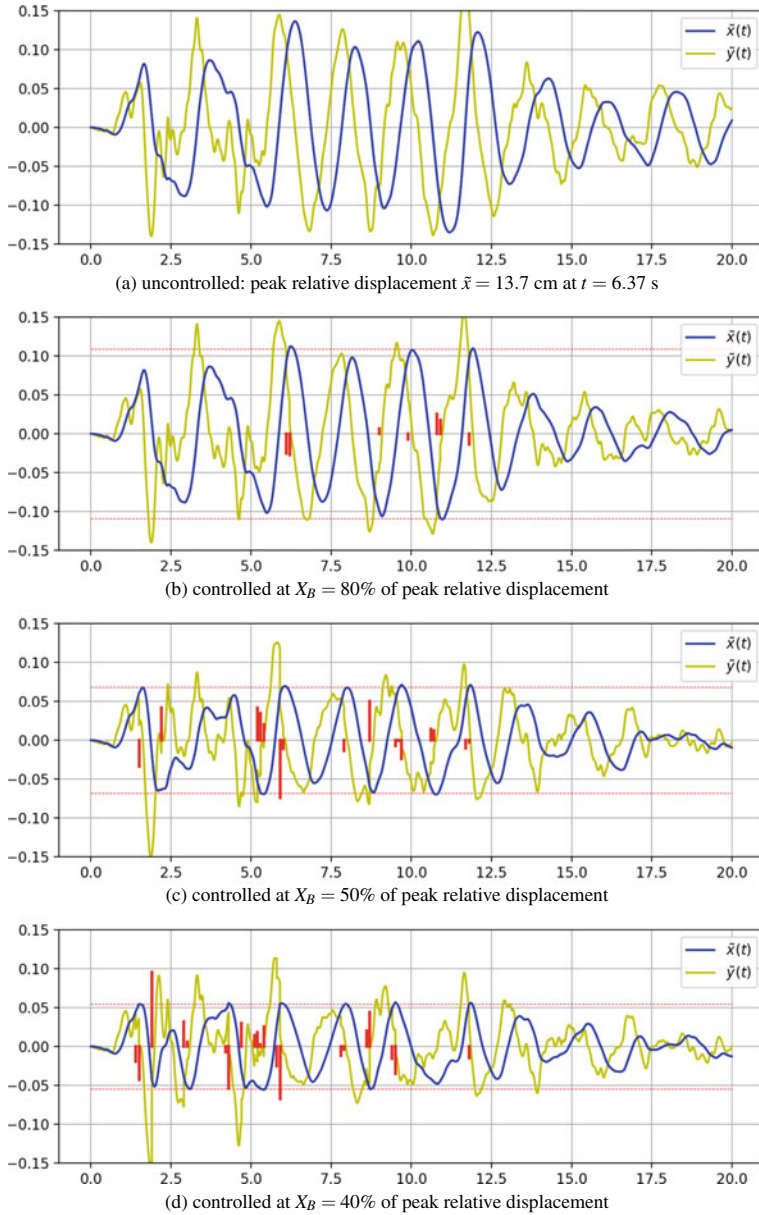


Fig. 18 Response to El Centro ground motion ($T_n = 2$ s, $\zeta = 5\%$, no time delay)

12 Conclusions

In this paper, we started by reviewing how the initial ideas of earthquake protection eventually crystallised into a number of distinct approaches, and we focused on the active control approach. We also reviewed Hybrid Event-B as a suitable vehicle for attempting a formal development of an SDOF active protection model. We then pursued the development through various levels of detail, culminating in the distributed sampled and quantized model of Sect. 9. Along the way, particularly in Sects. 8 and 10, we discussed the obstacles to accomplishing this with full formality.

In Sect. 11, we subjected our analytically derived model to simulation using well established numerical tools typically used in earthquake protection engineering. We spot-tested our model both on a simple harmonic excitation, and on the El Centro ground motion data. It was encouraging to see that our model behaved well, despite the relatively small input from the empirical sphere during its derivation. Enhancing the latter, can only be expected to improve matters regarding fidelity with conventional approaches.

The present study forms a launchpad for much possible future work. Firstly, there is the fact that our models' behaviour was timed with precision—in reality we will always have stochastic variations in the times of all events. Similar considerations apply to a better characterisation of the additional ground acceleration variability term e_B . Taking these issues into account would bring us closer to the level of detail of [16, 20, 21].

Secondly, there is the consideration of the replication of components needed for adequate fault tolerance. Here, at least, we can see that use of standard approaches would address the issue, and would again bring us closer to [16, 20, 21].

Thirdly, we note that the SDOF modelling can readily be enriched to capture the dynamics of a genuine building more accurately. The essentially scalar description we dealt with here could be enriched to encompass a greater number of linear and angular degrees of freedom. This again is relatively standard, at least in the linear dynamics case.

Fourthly, there is the investigation of richer formulations of retrenchment and refinement capable of coping with the wealth of phenomena discussed in Sect. 10. A generally applicable approach here would yield many dividends for a wide class of problems of a similar nature.

Fifthly, it is regrettable that there currently is no mechanised support for Hybrid Event-B. Nevertheless, progress with the issue just discussed would be a prerequisite for a meaningfully comprehensive coverage of the development route as a whole by mechanical means, even if individual parts could be treated by conventional mechanisation of linear and discrete reasoning. Taking all the above together, there is plenty to pursue in future work.

One final comment. In a recent UK terrestrial TV broadcast [25], various aspects of the construction of Beijing's Forbidden City were described. Not least among

these was the capacity of the Forbidden City's buildings to withstand earthquakes,¹⁹ particularly considering that Beijing lies in a highly seismic region. Fundamental to this is the use of bulky columns, which are essentially free standing, to support the weight of the building's heavy roof, and the use of complex dougong brackets [37, 38] to couple the columns to the roof. The free standing construction allows the ground under the building to slip during powerful tremors without breaking the columns, and the relatively flexible dougong brackets permit relative movement between the columns and other members without risking structural failure. These building techniques were already ancient by the time the Forbidden City was constructed early in the 1400s. The cited broadcast showed a scaled structure on a shaking table withstanding a simulated magnitude 10 quake. So, more recent efforts notwithstanding, the Chinese had the problem of earthquake protection for buildings licked more than two thousand years ago!

References

1. J. Earthq. Eng. Eng. Vib.
2. World Conferences on Earthquake Engineering
3. Abrial, J.R.: Modeling in Event-B: System and Software Engineering. Cambridge University Press, Cambridge (2010)
4. Ahmed, N.: Dynamic Systems and Control With Applications. World Scientific, Singapore (2006)
5. Banach, R.: Formal refinement and partitioning of a fuel pump system for small aircraft in Hybrid Event-B. In: Bonsangue D. (eds.) Proceedings of IEEE TASE-16, pp. 65–72. IEEE (2016)
6. Banach, R.: Hemodialysis machine in Hybrid Event-B. In: Butler, S., Mashkoo B. (eds.) Proceedings of ABZ-16. LNCS, vol. 9675, pp. 376–393. Springer (2016)
7. Banach, R.: The landing gear system in multi-machine Hybrid Event-B. Int. J. Softw. Tools Tech. Transf. **19**, 205–228 (2017)
8. Banach, R., Baugh, J.: Active earthquake control case study in Hybrid Event-B web site. <http://www.cs.man.ac.uk/~banach/some.pubs/EarthquakeProtection/>
9. Banach, R., Butler, M.: A Hybrid Event-B study of lane centering. In: Aiguier, B., Krob M. (eds.) Proceedings of CSDM-13, pp. 97–111. Springer (2013)
10. Banach, R., Butler, M.: Cruise control in Hybrid Event-B. In: Woodcock Z.L. (ed.) Proceedings of ICTAC-13. LNCS, vol. 8049, pp. 76–93. Springer (2013)
11. Banach, R., Butler, M., Qin, S., Verma, N., Zhu, H.: Core Hybrid Event-B I: single Hybrid Event-B machines. Sci. Comput. Program. **105**, 92–123 (2015)
12. Banach, R., Butler, M., Qin, S., Zhu, H.: Core Hybrid Event-B II: multiple cooperating Hybrid Event-B machines. Sci. Comput. Program. **139**, 1–35 (2017)
13. Banach, R., Jeske, C.: Retrenchment and refinement interworking: the tower theorems. Math. Struct. Comput. Sci. **25**, 135–202 (2015)
14. Banach, R., Poppleton, M., Jeske, C., Stepney, S.: Engineering and theoretical underpinnings of retrenchment. Sci. Comput. Program. **67**, 301–329 (2007)
15. Banach, R., Van Schaik, P., Verhulst, E.: Simulation and formal modelling of yaw control in a drive-by-wire application. In: Proceedings of FedCSIS IWCPs-15, pp. 731–742 (2015)

¹⁹Being wooden, the Forbidden City's buildings were less good at withstanding fire, and several structures have had to be rebuilt a number of times over the centuries because they had burnt down.

16. Baugh, J., Elseaidy, W.: Real-time software development with formal methods. *J. Comput. Civ. Eng.* **9**, 73–86 (1995)
17. Buckle, I.: Passive control of structures for seismic loads. In: *Proceedings of 12th World Conference on Earthquake Engineering*. Paper No. 2825 (2000)
18. Chicone, C.: *Ordinary Differential Equations with Applications*, 2nd edn. Springer, New York (2006)
19. Chopra, A.: *Dynamics of Structures: Theory and Applications to Earthquake Engineering*, 4th edn. Pearson, Englewood Cliffs (2015)
20. Elseaidy, W., Baugh, J., Cleaveland, R.: Verification of an active control system using temporal process algebra. *Eng. Comput.* **12**, 46–61 (1996)
21. Elseaidy, W., Cleaveland, R., Baugh, J.: Modeling and verifying active structural control systems. *Sci. Comput. Program.* **29**, 99–122 (1997)
22. Gattulli, V., Lepidi, M., Potenza, F.: Seismic protection of frame structures via semi-active control: modelling and implementation issues. *Earthq. Eng. Eng. Vib.* **8**, 627–645 (2009)
23. Gradshteyn, I., Ryzhik, I.: *Table of Integrals Series and Products*, 7th edn. Academic Press, New York (2007)
24. Kuramoto, H., Teshigawara, M., Okuzono, T., Koshika, N., Takayama, M., Hori, T.: Predicting the earthquake response of buildings using equivalent single degree of freedom system. In: *Proceedings of 12th World Conference on Earthquake Engineering*. Auckland, New Zealand. Paper No. 1039 (2000)
25. More4 TV: Secrets of China's Forbidden City. UK Terrestrial TV Channel: More4. (24 July 2017)
26. Olver, F., Lozier, D., Boisvert, R., Clark, C.: *NIST Handbook of Mathematical Functions*. Cambridge University Press, Cambridge (2010)
27. Platzer, A.: *Logical Analysis of Hybrid Systems: Proving Theorems for Complex Dynamics*. Springer, Berlin (2010)
28. Popescu, I., Sireteanu, T., Mitu, A.: A comparative study of active and semi-active control of building seismic response. In: *Proceedings of DGDS-09*, pp. 172–177. Geometry Balkan Press (2010)
29. Prucz, Z., Soong, T., Reinhorn, A.: An analysis of pulse control for simple mechanical systems. *J. Dyn. Syst. Meas. Control.* **107**, 123–131 (1985)
30. Retrenchment Homepage. <http://www.cs.man.ac.uk/~banach/retrenchment>
31. Rose, B., Baugh, J.: Parametric study of a pulse control algorithm with time delays. Technical report. CE-302-93, North Carolina State University Department of Civil Engineering (1993)
32. Sontag, E.: *Mathematical Control Theory*. Springer, New York (1998)
33. Soong, T.: *Active Structural Control: Theory and Practice*. Longman, Harlow (1990)
34. Soong, T., Chu, S., Reinhorn, A.: *Active, Hybrid and Semi-Active Control: A Design and Implementation Handbook*. Wiley, New York (2005)
35. Tabuada, P.: *Verification and Control of Hybrid Systems: A Symbolic Approach*. Springer, US (2009)
36. Walter, W.: *Ordinary Differential Equations*. Springer, Berlin (1998)
37. Wikipedia: Chinese architecture
38. Wikipedia: Dougong
39. Wikipedia: Duhamel's integral
40. Wikipedia: Earthquake engineering